AI Delivery Pods/Squads & Engineering-Quality KPI Framework

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

Solution Overview 4

What this is . . . 4

How it works . . . 5

Target model . . . 5

Benefits . . . 6

Limitations . . . 6

The deployment model: Assess → Design → Generate/Build → Validate → Expand . . . 6

Requirements 7

S1. One or more AI-delivery teams (or the mandate to form them) . . . 7

S2. Executive sponsorship for structure and measurement . . . 7

S3. Access to delivery telemetry (issue tracker, CI/CD, code review) . . . 8

S4. Access to the fidelity/privacy validation evidence . . . 8

S5. An agreed, small candidate KPI set (and the discipline to keep it small) . . . 8

S6. A dashboard / BI surface (or the agreement to use the portable feed) . . . 9

Global Requirements — Synthetic Data & AI Test-Enablement Solutions 9

1. A Python data/engineering environment . . . 9

2. A target clinical data model / standard . . . 10

3. An isolated non-production test environment . . . 10

4. Privacy, governance & compliance stakeholders and policy . . . 10

5. AI/QA use cases and a pilot scope . . . 10

6. Reference/real-data access for fidelity (read-only, minimized) . . . 11

Implementation 11

Stage 1 — Assess . . . 11

Stage 2 — Design . . . 12

Stage 3 — Generate/Build . . . 12

Stage 4 — Validate . . . 13

Stage 5 — Expand . . . 13

Testing & Validation 14

T1 — Structure-defined test (topology completeness) . . . 14

T2 — KPI-definition test (vanity-metric guard) . . . 14

T3 — Instrumentation test (data actually flows) . . . 14

T4 — Variance & target test . . . 15

T5 — Cadence-running test . . . 15

T6 — Privacy gate (non-negotiable) . . . 15

Overall acceptance . . . 16
Troubleshooting 16
The capacity planner flags a pod as over capacity . . . 16
The planner flags missing interfaces or single-person core roles . . . 16
The KPI validator rejects a KPI as a vanity metric . . . 17
A KPI has no data in the feed . . . 17
The rollup or privacy check aborts on a suspected-PHI column . . . 17
Variance looks "green" but the team knows quality is slipping . . . 17
The cadence produces reports but nothing changes . . . 18
Too many KPIs; the dashboard is ignored . . . 18
Best Practices 18
Structure . . . 18
Measurement . . . 19
Improvement . . . 19
Safety & governance . . . 19
Deliverables Summary 20
Documentation . . . 20
The 5 Python tools . . . 20
Appendix A — PowerShell Scripts 22

Executive Summary

Outcome. AI delivery organized for pace and proven for quality: delivery teams structured as pods and squads with clear roles and interfaces, and a small, honest set of KPIs — spanning engineering productivity and delivery quality — instrumented against targets, with an improvement cadence that turns variance into action. Leaders get to *see* whether AI delivery is getting better, not guess.

The problem it solves. Teams delivering AI at pace routinely get two things wrong: an unclear team structure (ownership and hand-offs are tribal, not written down) and the wrong measures (a dashboard full of vanity metrics — output counts, utilization, lines of code — that look busy but never move outcomes). The result is delivery that feels fast and fragile, with no credible way to prove quality or safety is holding.

The approach. The framework defines three layers and instruments all three. **Structure:** a pod/squad topology with named roles and explicit intake, hand-off, and escalation interfaces. **Measurement:** a curated catalog of 6–10 KPIs, each with a definition, data source, target, and owner, screened by an automated vanity-metric guard so raw counts and utilization never make the cut. **Improvement:** targets, a portable dashboard feed, and a recurring cadence where the largest gaps drive concrete, owned actions. It ships with five Python tools — a pod/squad capacity planner, a KPI-definition validator, a rollup/dashboard-feed generator, a target-vs-actual variance reporter, and a privacy checker — and follows a pilot-first **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** model with explicit pass/fail gates.

Why it's safe. The work it measures runs on **synthetic** data in **isolated, non-production** environments. The KPI pipeline consumes only operational telemetry and **aggregate, PHI-free** validation pass/fail signals — **never real Protected Health Information**, never patient-level detail. A non-negotiable privacy gate (with an automated checker) blocks expansion if any real PHI is ever found in the pipeline. Regulatory and de-identification *determinations* remain with the organization's privacy, compliance, and legal functions; the framework supplies the technical evidence, not the legal call. Crucially, privacy-gate first-pass rate is itself a first-class quality KPI — so "fast" can never quietly mean "unsafe".

The bottom line. Pods/squads structured, KPIs instrumented and de-vanitized, and an improvement cadence running — a self-guided framework, tested, at a fraction of the cost of a consulting engagement.

Solution Overview

What this is

This solution is an **operating model and measurement framework** for teams that deliver AI on synthetic clinical data and in safe, isolated test environments. It answers two questions that usually get answered badly: **how should AI-delivery teams be structured** (pods/squads, roles, and the interfaces between them), and **which KPIs actually prove that engineering productivity and delivery quality are improving** — as opposed to vanity metrics that look busy but change nothing.

The framework is deliberately lighter-weight than a data-generation solution: it does not build a hospital twin or generate a dataset. Instead it organizes the people who do, and instruments their work so leaders can see whether AI delivery is getting faster **and** better without cutting corners on privacy or safety. Every measurement input is either operational telemetry (tickets, pipelines, reviews) or the **fidelity/privacy validation evidence** that the synthetic solutions already produce — never real Protected Health

Information (PHI).

How it works

The framework has three layers, and this solution defines all three:

1. **Structure** — a pod/squad topology for AI delivery. A **pod** is the smallest end-to-end

delivery unit (a durable, cross-functional team that owns a slice of AI-delivery outcome). A **squad** is a coordinated grouping of pods working toward a shared program goal. Each pod has named **roles** (e.g. data/platform engineer, ML/AI engineer, QA/validation, privacy liaison, product/clinical SME, pod lead) and explicit **interfaces** — the contracts by which a pod receives work, hands off deliverables, and escalates. Structure is captured as the artifact `Pod Topology – Program – v1`.

1. **Measurement** — a curated KPI set spanning two families: **engineering productivity** (flow and

throughput of delivery) and **delivery quality** (defect escape, rework, validation pass rates, privacy-gate outcomes). Each KPI has a written **definition**, a **data source**, a **target**, and an **owner**, captured as `KPI Catalog – Program – v1`. The catalog is deliberately small; the framework guards against metric sprawl and against vanity metrics (raw output counts, utilization, lines of code) that do not correlate with outcomes.

1. **Improvement** — targets, dashboards, and a recurring **improvement cadence** (a regular

review where variance against targets drives concrete actions, not just reporting). The cadence artifact is `KPI Review Cadence – Program – v1`; the dashboard data model is `KPI Dashboard Feed – Program – v1`.

Because the underlying work runs on synthetic data in isolated environments, the KPI set includes a **safety/privacy dimension** as a first-class quality signal — for example, "share of releases that passed the privacy gate on first attempt" — so that speed is never rewarded at the expense of the non-negotiable privacy posture.

Target model

This is an operating-model/measurement solution, so its "target model" is a **measurement model**, not a clinical data model:

• **KPI definitions** follow a flow-and-quality frame (throughput, cycle time, defect escape,

rework, validation pass rate, privacy-gate first-pass rate). Where teams already use a recognized delivery-metrics vocabulary, the catalog maps onto it rather than replacing it — verify names against your organization's existing analytics conventions.

• **Data sources** are the delivery toolchain (issue tracker, CI/CD, code review, the

fidelity/privacy validation reports from the synthetic solutions). The framework specifies *what* to measure and *how to define it*; it references tool capabilities at the pattern level and defers vendor-console specifics to each tool's documentation.

• The KPI rollup and dashboard feed are expressed as **plain, portable data structures** (CSV/JSON)

so they can drive whatever BI/dashboard tool the organization already runs.

Benefits

• **A defensible team structure** — pods/squads, roles, and interfaces written down and agreed, so ownership and hand-offs are explicit instead of tribal.

• **KPIs that mean something** — a small, validated set tied to outcomes, with definitions and targets, instead of a dashboard full of vanity numbers.

• **Quality and privacy are visible, not assumed** — validation pass rates and privacy-gate outcomes sit alongside throughput, so "fast" can never quietly mean "unsafe".

• **An improvement engine** — a cadence that turns variance into action, so the numbers move because the team changes what it does.

• **Portable and tool-agnostic** — the rollup and dashboard feeds are standard data files; adopt them into your existing BI stack.

Limitations

• **This framework organizes and measures delivery; it does not perform it.** It assumes the synthetic-data/twin/AI-test work exists (or is being stood up) and instruments it.

• **KPIs are only as good as their inputs.** If the toolchain data is incomplete or inconsistent, the framework surfaces that as a data-quality gap — it does not fabricate numbers.

• **No regulatory determinations.** Privacy-gate outcomes are inputs to KPIs, but whether a given de-identification or disclosure-risk posture is *acceptable* remains a determination for the organization's privacy, compliance, and legal functions.

• **Targets are starting points.** Initial targets are conservative defaults; they must be re-baselined against your own history during the pilot.

The deployment model: Assess → Design → Generate/Build → Validate → Expand

This solution uses the same five-stage, pilot-first model as every solution in the category, mapped to an operating-model context:

• **Assess** — profile the current delivery reality: how many pods/squads exist (or are needed), what roles are staffed, what telemetry is available, and which metrics (if any) are in use. Output: `Delivery Assessment – Program – v1`.

• **Design** — define the target `Pod Topology – Program – v1`, the `KPI Catalog – Program – v1` (each KPI defined, sourced, targeted, owned), and the `KPI Review Cadence – Program – v1`.

• **Generate/Build** — instrument the KPIs: wire the data sources, build the `KPI Dashboard Feed – Program – v1`, and stand up the dashboard on the pilot squad.

• **Validate** — run the pass/fail gates: **structure defined** (topology + interfaces agreed),

KPIs instrumented (every catalog KPI has real data flowing and passes the vanity-metric guard), and **cadence running** (at least one improvement review completed with actions recorded). Privacy gate: no KPI input contains or exposes real PHI.

- **Expand** — extend from the pilot squad to additional squads, re-baselining targets and pruning

or adding KPIs based on what the pilot proved.

Each stage has an explicit pass/fail gate and a documented rollback (revert to the prior catalog/ topology version) so the framework can be adopted safely and incrementally.

Requirements

This framework builds on the **six category-wide requirements** in `global-requirements-synthetic.md` (a Python data/engineering environment, a target clinical data model, an isolated non-production test environment, privacy/governance stakeholders and policy, AI/QA use cases with a pilot scope, and read-only/minimized reference-data access for fidelity). Confirm those first — in particular that **no real PHI ever enters a shared harness, twin, or AI test environment**, and that regulatory/de-identification determinations remain with the organization's privacy, compliance, and legal functions.

The prerequisites below are **specific to this operating-model/measurement solution**. Each is listed with its Description, Why required, How to verify, and How to implement if missing.

S1. One or more AI-delivery teams (or the mandate to form them)

- **Description:** At least one team that delivers AI on synthetic data / in isolated test

environments — the people who will be organized into the pod/squad topology and whose work the KPIs measure. For a greenfield program, the executive **mandate** to form such teams.

- **Why required:** The framework structures and measures delivery teams; there must be delivery to

structure and measure.

- **How to verify:** Confirm with the sponsor which team(s) or program(s) are in scope for the

pilot, and roughly how many people and roles exist today.

- **How to implement if missing:** If no team exists yet, agree the pilot squad's staffing and

charter with the sponsor before designing the topology.

S2. Executive sponsorship for structure and measurement

- **Description:** A named executive sponsor who owns the operating-model change and will act on KPI

outcomes (re-prioritize, staff, or unblock) rather than treat the dashboard as reporting theater.

- **Why required:** Structure changes (who owns what) and measurement changes (which KPIs are

"official") are organizational decisions; without a sponsor the cadence produces reports no one acts on, and the metrics decay into vanity numbers.

- **How to verify:** Confirm the sponsor by name and that they will chair or attend the improvement

cadence.

• **How to implement if missing:** Secure sponsorship before publishing the KPI catalog; an un-sponsored catalog should not be declared "official".

S3. Access to delivery telemetry (issue tracker, CI/CD, code review)

• **Description:** Read access to the operational data sources the productivity and quality KPIs are computed from — the issue/work tracker, the CI/CD pipeline history, and the code-review system — exported as data (CSV/JSON) or via their APIs.

• **Why required:** KPIs such as cycle time, throughput, defect escape, and rework are computed from this telemetry. Without it the framework cannot instrument the catalog.

• **How to verify:** Confirm you can export, for the pilot squad, a work-item history (with created/closed timestamps and type), a build/deploy history (with pass/fail), and a review history.

• **How to implement if missing:** Arrange read-only exports or API access with the tool owners before the Generate/Build stage. Reference each tool's documentation for export specifics; do not assume console internals — verify against your version.

S4. Access to the fidelity/privacy validation evidence

• **Description:** The **validation reports** the synthetic solutions already produce — fidelity/coverage results and privacy-gate outcomes — as the source for the quality KPIs' safety dimension.

• **Why required:** The framework treats "did the release pass its privacy gate" and "did synthetic output meet its fidelity threshold" as first-class quality signals, so speed is never rewarded over safety. These come from the validation evidence, not from operational telemetry.

• **How to verify:** Confirm you can obtain, per release/dataset, a pass/fail fidelity result and a pass/fail privacy-gate result. **These are aggregate pass/fail signals — never the underlying records, and never real PHI.**

• **How to implement if missing:** Agree with the validation owners a standard, PHI-free pass/fail export before defining the quality KPIs.

S5. An agreed, small candidate KPI set (and the discipline to keep it small)

• **Description:** A shortlist of candidate KPIs covering both productivity (flow/throughput) and quality (defect/rework/validation/privacy), with an explicit agreement to keep the official set small (roughly 6–10) and to **exclude vanity metrics** (raw output counts, individual utilization, lines of code, story-point velocity used as a productivity target).

• **Why required:** Metric sprawl and vanity metrics are the primary failure mode of KPI programs; the framework's value depends on a curated, outcome-tied set.

• **How to verify:** Confirm stakeholders agree to the small-set principle and the vanity-metric

exclusion list before the catalog is authored.

• **How to implement if missing:** Facilitate a short definition workshop; use the KPI-definition validator tool (see deliverables) to screen candidates against the vanity-metric guard.

S6. A dashboard / BI surface (or the agreement to use the portable feed)

• **Description:** A place to render the KPIs — an existing BI/dashboard tool, or agreement to consume the framework's portable `KPI Dashboard Feed – Program – v1` (CSV/JSON) directly.

• **Why required:** The improvement cadence needs the KPIs visible against targets; the framework produces a tool-agnostic feed rather than binding to one BI product.

• **How to verify:** Confirm which surface the pilot squad will use and that it can ingest CSV/JSON.

• **How to implement if missing:** If no BI tool is available, the cadence can run directly off the generated feed files and the variance report until a dashboard is provisioned.

Global Requirements — Synthetic Data & AI Test-Enablement Solutions

These requirements apply to **every** solution in the **Synthetic** category (synthetic clinical data, hospital-twin simulation, and safe AI test enablement). Verify each before starting a solution's steps.

> ***Safety, vendor & regulatory boundary (applies to every Synthetic solution).*** *These solutions generate and validate **synthetic** (artificial) data and run in **isolated non-production** test environments. **Never place real Protected Health Information (PHI) into a shared synthetic harness, twin, or AI test environment** — where real data is touched at all it is read-only, minimized, and handled under the governing privacy controls. **Synthetic-data and digital-twin tools** (e.g. Synthea, and commercial platforms) and **AI/model frameworks** are integrated at the pattern/architectural level; specific vendor console internals are **cited or deferred** to that vendor's documentation, never invented. **Regulatory and legal interpretation** (HIPAA/HITRUST de-identification determinations, IRB/privacy-office sign-off, disclosure risk acceptance, and any ITAR/NERC/NRC or contract constraints) is **deferred to the organization's privacy, compliance, and legal functions** — the solutions provide the technical method and evidence, not legal determinations.*

1. A Python data/engineering environment

• **Description:** A supported **Python** environment (e.g. 3.10+) with the standard data toolchain (data frames, statistics, and the synthetic-data / validation libraries the solution uses).

• **Why required:** The automation and reusable assets are authored in Python (generation, de-identification, fidelity/utility metrics, scenario runners).

• **How to verify:** Confirm Python + the required packages install and run in the target environment.

• **How to implement if missing:** Provision the environment and install dependencies before starting.

2. A target clinical data model / standard

• **Description:** A defined **clinical data model or interoperability standard** to generate against — e.g. **FHIR**, **OMOP CDM**, or the source system's own schema — plus the fields/domains in scope (patients, encounters, conditions, medications, labs, vitals, notes, etc.).

• **Why required:** Realistic synthetic data and a credible hospital twin require a target schema and clinically plausible relationships; validation is measured against it.

• **How to verify:** Confirm the target model/standard and the in-scope domains with the AI/QA teams.

• **How to implement if missing:** Select and document the target model before generating.

3. An isolated non-production test environment

• **Description:** A **segregated, non-production** environment (compute + storage) for the synthetic harness, hospital twin, and AI test runs — separate from production and from real-PHI systems.

• **Why required:** Safe evaluation of AI solutions before broader adoption depends on isolation; it also bounds any residual disclosure risk.

• **How to verify:** Confirm the environment is isolated, access-controlled, and not connected to production PHI stores.

• **How to implement if missing:** Stand up / obtain the isolated environment before running.

4. Privacy, governance & compliance stakeholders and policy

• **Description:** The governing **privacy, data-governance, and compliance** policies and the named stakeholders (privacy officer, security, and — where real source data or de-identification claims are involved — IRB/legal) whose requirements the work must satisfy.

• **Why required:** Regulated healthcare data constraints, de-identification acceptance, and enterprise data governance are policy decisions the technical method must align to, not defaults.

• **How to verify:** Confirm the applicable policies, disclosure-risk threshold, and approvers.

• **How to implement if missing:** Identify the policies and stakeholders before generating or releasing test data.

5. AI/QA use cases and a pilot scope

• **Description:** The **AI use cases and QA scenarios** the synthetic data / twin must support, and a **pilot scope** to validate on before broader rollout.

• **Why required:** Fidelity, coverage, and scenario design are driven by the AI use cases; every solution follows an **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** model proven on a pilot.

• **How to verify:** Confirm the target AI use cases, evaluation goals, and a named pilot.

• **How to implement if missing:** Elicit the use cases and define the pilot before building the harness.

6. Reference/real-data access for fidelity (read-only, minimized)

• **Description:** Where synthetic **fidelity/utility** must be measured against reality, **read-only, minimized** access to reference statistics or a governed real dataset (under the privacy controls in §4) — or agreed reference distributions where real data cannot be used.

• **Why required:** Fidelity and validation assets compare synthetic output to real-world distributions and relationships; without a reference the claims are unfounded.

• **How to verify:** Confirm the reference source and its access/governance terms.

• **How to implement if missing:** Agree the reference approach (governed real stats vs. published reference distributions) before validating.

*The deployment model across every Synthetic solution is the same: **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand**, with explicit pass/fail (fidelity/privacy) gates and a documented rollback. Automation and reusable assets are delivered as **Python** — never real PHI is written to a shared harness, twin, or AI test environment.*

Implementation

Follow these steps in order. They implement the **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** model, pilot-first. Every artifact is named with the `<Purpose> – <Scope> – <Stage>` convention and that exact name is reused in later steps, testing, and the scripts. Run everything on a **single pilot squad** first; do not roll out program-wide until the Validate gates pass.

Throughout: KPI inputs are operational telemetry and **aggregate, PHI-free** validation pass/fail signals only. **Never** feed real PHI, patient-level records, or synthetic-but-sensitive detail into a KPI pipeline or dashboard feed.

Stage 1 — Assess

Goal: capture the current delivery reality before changing anything.

1. **Confirm scope and sponsor.** With the executive sponsor (requirement S2), fix the **pilot squad** and its charter. Record it at the top of the assessment.

1. **Inventory the current structure.** List the teams/pods in the pilot scope today, the roles staffed, and the informal interfaces (how work arrives, how it hands off, how issues escalate).

1. **Inventory the telemetry.** For the pilot squad, confirm you can export a work-item history, a CI/CD history, a code-review history (S3), and the fidelity/privacy validation pass/fail signals (S4). Note any gaps.

1. **Inventory existing metrics.** List any KPIs/dashboards already in use and flag which are vanity metrics (raw counts, individual utilization, LOC, velocity-as-target).

1. **Produce** `Delivery Assessment – Program – v1` — a short document capturing the above: current

structure, telemetry availability (with gaps), and existing-metric findings. **Gate to proceed:** telemetry for the pilot squad is available or has an owner committed to providing it.

Stage 2 — Design

Goal: define the target structure, the KPI catalog, and the improvement cadence.

1. **Define the pod topology.** Using the config helper (`pod_capacity_planner.py`), model the target

pods for the pilot squad: for each pod, its outcome ownership, its roles (data/platform engineer, ML/AI engineer, QA/validation, privacy liaison, product/clinical SME, pod lead), and its **interfaces** (intake contract, hand-off/deliverable contract, escalation path). Confirm each pod is staffable and not over-subscribed. Record as `Pod Topology – Program – v1`.

1. **Draft the candidate KPI set.** In a short workshop (S5), draft 6–10 candidate KPIs spanning:

• **Productivity/flow:** delivery cycle time, throughput (items completed per period), work-in-

progress, flow efficiency.

• **Quality:** defect escape rate, rework rate, change-failure rate, fidelity validation

first-pass rate, **privacy-gate first-pass rate**.

1. **Validate the KPI definitions.** Run `kpi_definition_validator.py` over the candidate catalog.

Every KPI must have a **definition, data source, target, owner**, and must **pass the vanity-metric guard** (no raw output counts, individual utilization, LOC, or velocity-as-target). Fix or drop any KPI that fails. Record the passing set as `KPI Catalog – Program – v1`.

1. **Set targets.** For each KPI, set a **conservative starting target** and mark it "provisional —

re-baseline during pilot". Targets are directional at this stage; the pilot produces the real baseline.

1. **Define the cadence.** Document `KPI Review Cadence – Program – v1`: who attends (sponsor +

pod leads), frequency (e.g. every two weeks), and the standing agenda — review variance, pick the one or two largest gaps, agree concrete actions with owners, and record them. **Gate to proceed:** topology, catalog (validator-passed), and cadence are all agreed and signed off.

Stage 3 — Generate/Build

Goal: instrument the KPIs and stand up the dashboard for the pilot squad.

1. **Wire the data sources.** Export the pilot squad's telemetry and validation pass/fail signals

into the framework's input format (a per-KPI observations file). Use each tool's documented export path; verify field names against your version rather than assuming.

1. **Compute the rollup.** Run `kpi_rollup_generator.py` to aggregate the raw observations into

per-KPI, per-period values, producing `KPI Dashboard Feed – Program – v1` (CSV/JSON). This is the single, portable feed the dashboard and the cadence consume.

1. **Stand up the dashboard.** Point your BI tool (or, if none, the cadence itself) at `KPI Dashboard Feed – Program – v1`. Confirm every catalog KPI renders against its target.
1. **Run the first variance report.** Run `kpi_variance_reporter.py` against the feed and the catalog targets to produce the target-vs-actual variance for the pilot period. This is the input to the first cadence review. **Gate to proceed:** every catalog KPI has real data flowing and a computed value; no feed field contains PHI.

Stage 4 — Validate

Goal: pass the three framework gates before expanding. (Full procedures in `05-testing.md`.)

1. **Structure-defined gate.** `Pod Topology – Program – v1` exists, every pod has roles and all three interfaces (intake, hand-off, escalation), and no pod is over-subscribed per the capacity planner. **Pass/fail:** any pod missing an interface or over capacity = fail.
1. **KPIs-instrumented gate.** Every KPI in `KPI Catalog – Program – v1` passes the validator (definition/source/target/owner + vanity guard) **and** has a real value in `KPI Dashboard Feed – Program – v1`. **Pass/fail:** any KPI failing the validator or with no data = fail.
1. **Cadence-running gate.** At least one improvement review has run off the variance report, with actions recorded and owners assigned in `KPI Review Cadence – Program – v1`. **Pass/fail:** no completed review with recorded actions = fail.
1. **Privacy gate (non-negotiable).** No KPI observation, rollup, or dashboard feed contains real PHI or patient-level detail — only aggregate metrics and PHI-free pass/fail signals. **Pass/fail:** any real PHI present = fail and stop; remediate before proceeding.

Rollback: if any gate fails and cannot be remediated in place, revert to the prior version of the affected artifact (`… – v1` → prior) and re-run the stage. Do not expand on a failed gate.

Stage 5 — Expand

Goal: extend beyond the pilot, having proven the framework.

1. **Re-baseline targets.** Using the pilot's actuals, replace the provisional targets in `KPI Catalog – Program – v1` with evidence-based targets. Bump to `KPI Catalog – Program – v2`.
1. **Prune and add KPIs.** Drop any KPI the pilot showed to be noise or a de-facto vanity metric; add any genuinely missing outcome signal (re-run the validator on the changed set).
1. **Onboard the next squad.** Repeat Stages 2–4 for the next squad, reusing the topology pattern and the validated catalog. Version per squad/program as `… – v2`, `… – v3`.

1. **Institutionalize the cadence.** Make the improvement cadence a standing program ritual chaired by the sponsor, with the variance report as its standing input.

Testing & Validation

These procedures validate the framework before it is trusted and expanded. They map to the three framework gates plus the non-negotiable privacy gate. Run every test on the **pilot squad** and record the result in `Delivery Assessment – Program – v1` (or a linked validation record). Each test states its expected **pass/fail** signal.

Throughout, remember the safety boundary: KPI inputs are operational telemetry and **aggregate, PHI-free** validation pass/fail signals only. Any test that would require patient-level data to run is out of scope by design.

T1 — Structure-defined test (topology completeness)

Purpose: confirm `Pod Topology – Program – v1` is complete and staffable.

Procedure:

1. For each pod, confirm it has: an outcome it owns, at least the core roles, and all three interfaces (intake contract, hand-off/deliverable contract, escalation path).
1. Run `pod_capacity_planner.py` on the topology and confirm no pod is over-subscribed (allocated demand ≤ capacity) and no role is single-points-of-failure beyond the agreed threshold.

Expected result / pass-fail: Pass = every pod has an owned outcome, core roles, and all three interfaces, and no pod exceeds capacity. **Fail** = any pod missing an interface, missing a core role, or over capacity.

T2 — KPI-definition test (vanity-metric guard)

Purpose: confirm `KPI Catalog – Program – v1` contains only well-formed, non-vanity KPIs.

Procedure:

1. Run `kpi_definition_validator.py` over the catalog.
2. Confirm every KPI has a definition, a data source, a target, and an owner.
3. Confirm the vanity-metric guard flags **zero** KPIs (no raw output counts, individual utilization, lines of code, or velocity-as-target).

Expected result / pass-fail: Pass = validator reports every KPI complete and zero vanity flags. **Fail** = any missing field or any vanity flag. Remediate the flagged KPI (redefine or drop) and re-run.

T3 — Instrumentation test (data actually flows)

Purpose: confirm every catalog KPI has real data in the feed.

Procedure:

1. Run `kpi_rollup_generator.py` on the pilot observations to (re)build

`KPI Dashboard Feed – Program – v1`.

1. Confirm every KPI in the catalog appears in the feed with a non-null value for the current period.

2. Confirm counts reconcile against a manual spot-check of one KPI (e.g. recompute cycle time for a

handful of items by hand).

Expected result / pass-fail: Pass = every catalog KPI has a value in the feed and the spot-check reconciles within tolerance. **Fail** = any KPI missing a value, or the spot-check diverges materially (investigate the data source mapping).

T4 — Variance & target test

Purpose: confirm targets are set and variance is computed correctly.

Procedure:

1. Run `kpi_variance_reporter.py` against `KPI Dashboard Feed – Program – v1` and the catalog targets.

2. Confirm each KPI shows actual, target, variance, and a direction-aware status (a lower-is-better

KPI like cycle time must not be flagged "good" when it rises).

Expected result / pass-fail: Pass = variance and status are correct for both higher-is-better and lower-is-better KPIs, verified on at least one of each. **Fail** = any direction handled incorrectly or any target missing.

T5 — Cadence-running test

Purpose: confirm the improvement loop actually runs and produces action, not just reporting.

Procedure:

1. Hold the first review per `KPI Review Cadence – Program – v1`, using the T4 variance report as

input.

1. Confirm the review selected the one or two largest gaps, agreed concrete actions, and assigned

owners, all recorded.

Expected result / pass-fail: Pass = at least one completed review with recorded actions and owners. **Fail** = review not held, or held with no recorded actions/owners.

T6 — Privacy gate (non-negotiable)

Purpose: confirm no real PHI is present anywhere in the KPI pipeline.

Procedure:

1. Run `kpi_privacy_check.py` over the observations file and `KPI Dashboard Feed – Program – v1`.
2. Confirm the checker finds no patient-level identifiers, free-text clinical detail, or fields that

look like PHI — only aggregate metrics and PHI-free pass/fail signals.

1. Confirm the validation inputs (S4) were the aggregate pass/fail signals, not underlying records.

Expected result / pass-fail: Pass = zero suspected-PHI findings; inputs confirmed aggregate. **Fail** = any suspected PHI or patient-level field present → **stop**, remove it, and re-run before proceeding. A privacy-gate failure blocks expansion unconditionally.

Overall acceptance

The framework is validated for the pilot when **T1–T6 all pass**. Only then proceed to Stage 5 (Expand). Any failed gate triggers the documented rollback (revert the affected … – `v1` artifact and re-run its stage). Determinations about whether the privacy posture is *legally acceptable* remain with the organization's privacy, compliance, and legal functions — these tests provide technical evidence, not a determination.

Troubleshooting

Common failure modes when standing up the pod/squad structure and the KPI framework, and how to resolve each. Symptoms map to the gates in `05-testing.md`.

The capacity planner flags a pod as over capacity

Symptom: `pod_capacity_planner.py` reports `over_capacity=True` / RESULT: FAIL for a pod in `Pod Topology – Program – v1`.

Likely causes & resolution:

• The pod owns too broad a slice — split the outcome so each pod is end-to-end and self-sufficient.

• Demand is temporarily spiked — re-run with the sustainable demand, not a one-off surge; if the

spike is real, staff the pod or defer scope.

• Capacity is under-recorded — confirm the capacity unit and the allocations are right (the planner

is only as good as the numbers you feed it).

• Do **not** "fix" this by raising the recorded capacity to make the flag disappear — that just hides

the over-subscription and it will resurface as poor quality metrics.

The planner flags missing interfaces or single-person core roles

Symptom: `MISSING interfaces` or `single-person core roles (SPOF risk)` in the planner output.

Resolution: Every pod needs all three interfaces (intake, hand-off, escalation) written down — add the missing contract to the topology. For SPOF roles (a core role staffed by exactly one person), cross-train or add coverage; at minimum, record the risk and a mitigation. This is the **structure-defined gate (T1)** — it must pass before instrumenting KPIs.

The KPI validator rejects a KPI as a vanity metric

Symptom: `kpi_definition_validator.py` prints `VANITY: …` for a candidate KPI.

Resolution: This is working as designed. Raw output counts, lines of code, individual utilization, and velocity-as-target are excluded because they reward the wrong behavior. **Redefine** the metric around an outcome (e.g. replace "commits per sprint" with "delivery cycle time" or "throughput of completed items") or **drop** it. Re-run the validator until zero vanity flags remain. Do not edit the guard to smuggle a vanity metric through.

A KPI has no data in the feed

Symptom: `kpi_rollup_generator.py` produces a feed missing a catalog KPI, or the instrumentation test (T3) shows a null value.

Likely causes & resolution:

• The data source isn't wired — confirm the telemetry export (S3) or validation signal (S4) for that KPI is actually flowing into the observations file.

• Field-name mismatch — the tool's export column names differ from what you mapped; verify against your tool version's documentation (don't assume console internals).

• The KPI can't be sourced at all — if no data source exists, the KPI is aspirational; drop it from `KPI Catalog – Program – v1` until a source exists, rather than leaving a permanent blank.

The rollup or privacy check aborts on a suspected-PHI column

Symptom: `kpi_rollup_generator.py` aborts with "suspected-PHI column", or `kpi_privacy_check.py` returns FAIL.

Resolution: Stop. A KPI feed must contain only aggregate, PHI-free metrics. Remove the offending column or value **at the source** (you are almost certainly exporting raw records instead of aggregates), then re-export and re-run. This is the **non-negotiable privacy gate (T6)** — expansion is blocked until it passes. Remember the checker is a technical screen; whether any residual posture is *acceptable* remains a determination for privacy/compliance/legal.

Variance looks "green" but the team knows quality is slipping

Symptom: `kpi_variance_reporter.py` shows KPIs on target, yet defects or privacy issues are rising in reality.

Likely causes & resolution:

• **Direction handled wrong** — confirm each KPI's `direction`. A lower-is-better KPI (cycle time, defect escape) must be flagged when it *rises*; the reporter is direction-aware, so a green status on a rising bad metric means the direction is mis-set in the catalog.

• **Gaming** — a productivity KPI with no paired quality KPI invites shortcuts. Ensure every productivity KPI is paired with a quality KPI (see Best Practices); add the missing pairing.

• **Wrong or stale target** — provisional targets not re-baselined can read "green" while masking regression. Re-baseline from real history at Stage 5.

The cadence produces reports but nothing changes

Symptom: reviews happen, but the same variances persist cycle after cycle (cadence-running gate T5 technically passes, but improvement stalls).

Resolution: The cadence exists to drive **action**, not reporting. Enforce the standing agenda: pick the one or two largest gaps, agree a concrete action, assign a named owner, and check it next cycle. If actions aren't being done, escalate to the sponsor (S2) — an un-actioned cadence is a sponsorship problem, not a metrics problem.

Too many KPIs; the dashboard is ignored

Symptom: the catalog has grown well past 6–10 and no one looks at the dashboard.

Resolution: Prune. The validator prints a NOTE when the set exceeds ten. Retire any KPI that hasn't driven a decision in the last couple of cycles. A small, trusted set beats a large, ignored one. Bump the catalog version after pruning.

Best Practices

Durable practices for running an AI-delivery pod/squad structure and an engineering-quality KPI framework that stays honest over time.

Structure

• **Keep pods small and end-to-end.** A pod should own a slice of outcome it can deliver without constant cross-team dependency. If a pod can't ship without three other teams, the boundary is wrong — redraw it.

• **Write the interfaces down.** The intake contract, the hand-off/deliverable contract, and the escalation path are the parts that fail silently. Capture them in `Pod Topology – Program – v1` and review them when hand-offs go wrong.

• **Name a privacy liaison in every pod.** Because the work runs on synthetic data in isolated environments, someone in each pod owns the privacy posture and the privacy gate — it is not a central afterthought.

• **Don't over-subscribe pods.** Use the capacity planner every time demand or staffing changes; a chronically over-capacity pod produces bad quality metrics that no cadence can fix.

Measurement

• **Keep the set small (6–10) and outcome-tied.** Every KPI must plausibly connect to "AI delivery is faster and safer". If you can't state the outcome a metric serves, drop it.

• **Ban vanity metrics explicitly.** Raw output counts, individual utilization, lines of code, and velocity-as-a-target reward the wrong behavior. Keep the exclusion list in the catalog and run the validator on every catalog change.

• **Pair every productivity KPI with a quality KPI.** Throughput without defect-escape, or cycle time without rework, invites gaming. The pairing is what keeps "fast" honest.

• **Treat privacy-gate first-pass rate as a first-class quality KPI.** It is the metric that makes the safety boundary visible; never let a speed metric out-rank it.

• **Never measure individuals.** KPIs are team/pod-level. Individual metrics erode trust and drive local optimization that hurts the system.

• **Define once, source once.** Each KPI has exactly one written definition and one data source. Duplicate or ambiguous definitions are how dashboards start to lie.

Improvement

• **Variance drives action, not reporting.** The cadence exists to pick the one or two biggest gaps and do something about them, with a named owner. A review that only "notes" variance is theater.

• **Re-baseline targets from real history.** Provisional targets are placeholders; replace them with evidence from the pilot and revisit them each quarter.

• **Prune ruthlessly.** If a KPI hasn't driven a decision in a couple of cycles, it is probably noise — retire it. A smaller, trusted set beats a large, ignored one.

• **Keep the human context.** Numbers show *where* to look; the pod's judgment explains *why*. Pair every flagged variance with a short narrative before acting.

Safety & governance

• **Aggregate in, aggregate out.** Only aggregate metrics and PHI-free pass/fail signals enter the pipeline. Run the privacy checker on the feed as a standing gate, not just once.

• **Version every artifact.** Topology, catalog, cadence, and feed all carry `– v1`, `– v2`… so a

bad change can be rolled back cleanly.

• **Defer the determinations.** The framework produces evidence (pass rates, variance); whether a privacy posture is legally acceptable stays with privacy/compliance/legal. Don't let a green dashboard be mistaken for a compliance sign-off.

Deliverables Summary

Everything the buyer receives, on one page.

Documentation

• **Solution Overview** — the AI-delivery operating model: pods/squads, roles and interfaces, and the productivity + delivery-quality measurement approach, plus the Assess → Design → Generate/Build → Validate → Expand model.

• **Requirements Guide** — the six category-wide prerequisites plus six solution-specific ones (delivery teams, sponsorship, telemetry access, validation-evidence access, a small candidate KPI set, and a dashboard surface), each with how to verify and how to implement.

• **Implementation Guide** — exact ordered steps: assess current delivery, design the pod topology and KPI catalog, instrument the KPIs and dashboard feed, validate against the three gates, and expand — pilot-first, with every artifact named and every pass/fail gate stated.

• **Testing & Validation** — six procedures (structure-defined, KPI-definition/vanity-guard, instrumentation, variance/target, cadence-running, and the non-negotiable privacy gate), each with its expected pass/fail signal.

• **Troubleshooting** — the common failure modes (missing telemetry, vanity-metric creep, gamed metrics, off-target with no action, suspected PHI in the feed) and how to resolve each.

• **Best Practices** — durable practices for structure, measurement, improvement, and safety — including keeping the KPI set small and banning vanity metrics.

• **Executive Summary** — the outcome, approach, and why it is safe, for sponsors.

The 5 Python tools

1. `pod_capacity_planner.py` — models the pod/squad topology (pods, roles, interfaces, allocations, demand) and reports whether each pod is staffable and within capacity, flagging missing interfaces and single-role points of failure. Produces `Pod Topology – Program – v1` inputs. Read-only unless `--out` is passed.

1. `kpi_definition_validator.py` — validates the KPI catalog: every KPI must have a definition, data source, target, owner, and direction, and must pass the **vanity-metric guard** (rejects raw output counts, lines of code, individual utilization, velocity-as-target). Gate for `KPI Catalog – Program – v1`.

1. `kpi_rollup_generator.py` — aggregates raw, PHI-free KPI observations into the portable

`KPI Dashboard Feed – Program – v1` (CSV/JSON), rejecting any suspected-PHI column. Dry-run by default; writes only with `--out`.

1. `kpi_variance_reporter.py` — joins the dashboard feed to catalog targets and reports

direction-aware target-vs-actual variance (a lower-is-better KPI is never "good" when it rises). The standing input to the improvement cadence.

1. `kpi_privacy_check.py` — the **privacy gate**: read-only scan of the observations/feed for

patient-level column names and PHI-shaped cell patterns (MRN/SSN/email/phone/DOB/free-text), reporting counts only (never echoing values). PASS/FAIL blocks expansion.

All tools are Python 3.10+, standard-library-first, safe-by-default (no real PHI, dry-run/opt-in for writes, no network calls, no hardcoded secrets), and self-documenting via `--help` and `--emit-sample`.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

kpi_definition_validator.py

kpi_privacy_check.py

kpi_rollup_generator.py

kpi_variance_reporter.py

pod_capacity_planner.py